IR INFOTECH

E-7 OPERATIONS ENGINE

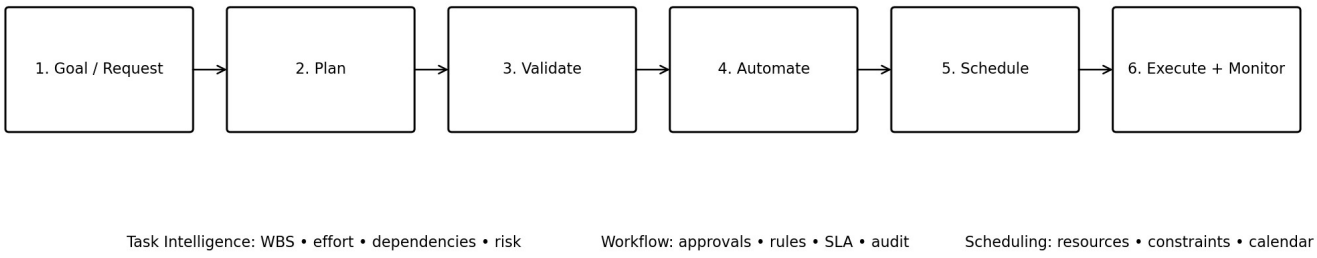
Complete Product, Research, Architecture & Implementation Specification

Unified from Task Intelligence Engine (2) + Workflow Engine (13) + Scheduling Engine (24)

Prepared from the supplied IR Main Document, Consolidated Engine Directory, Engine Architecture, three source-engine stories, and the supplied E-7 implementation plan.

Version: 1.0 | Date: 15 September 2026 | Status: Implementation Blueprint

Plan → Automate → Schedule → Monitor Loop



Important interpretation: the request mentions “observation engine” once, but the assigned engine and all supplied E-7 material identify the target as the Operations Engine. This document therefore specifies E-7 Operations Engine.

1. Executive Summary

The E-7 Operations Engine is the orchestration layer for work, people, time and institutional processes. It combines three previously separate capabilities into one coherent Plan → Automate → Schedule pipeline: Task Intelligence converts goals into executable work; Workflow executes the process using deterministic rules, approvals and SLAs; Scheduling solves when, where and with which resources the work should occur.

Source basis: *Consolidated Engine Directory defines the merge and the Plan → Automate → Schedule unifying logic; Engine Architecture describes E-7 as orchestration of work, time and institutional processes.*

The most important architectural rule is separation of probabilistic AI reasoning from deterministic execution. LLMs are used for goal interpretation, WBS generation, recommendation and explanation. Business-process state transitions, approval rules, SLAs and scheduling constraints must remain deterministic. The supplied implementation plan explicitly recommends this split to preserve reliability, scalability and compliance.

The engine must be API-first and support two access modes: Universal Mode for general-purpose standalone use, and Embedded/Integrated Mode where tenant data, permissions, resources, calendars and ERP entities are available inside the client data boundary.

2. Source Analysis and Reconciled Scope

Seven supplied documents were analyzed. The source material is internally consistent on the core objective, but it exists at different levels: original engine stories, overall platform requirements, consolidated architecture, and a short E-7 implementation plan. The detailed sections below preserve source requirements and explicitly label additional engineering details as proposed where the source does not specify them.

Source	What it contributes to E-7
IR MAIN DOC(10)	Mandatory market-first research process; platform goals; 9-dimension evaluation framework; implementation lifecycle; context/memory/twin/event-driven principles; original specifications for engines 2, 13 and 24.
Consolidated Engine Directory(5)	Formal merge into E-7; unifying logic; consolidated features; target outputs.
Engine Architecture - IR Infotech AI Ecosystem(4)	Non-negotiable design philosophy, dual access model, detailed E-7 modes, deterministic scheduling principle.
2task_intelligence(1)	Story, business value, features, deliverables, acceptance criteria for Task Intelligence.
13workflow(1)	Story, business value, features, deliverables, acceptance criteria for Workflow.
19scheduling(1)	Story, business value, features, deliverables, acceptance criteria for Scheduling.
E-7 Comprehensive Implementation Plan	Reasoning/execution split; candidate tools; decoupled microservices; five implementation phases.

3. Product Definition

3.1 What the Operations Engine Is

E-7 is not a project-management UI, a workflow tool, or a timetable generator by itself. It is an intelligence and orchestration layer that can be embedded into existing systems through APIs or exposed through IR Infotech's universal interface. Its responsibility is to accept an operational objective, convert it into a machine-manageable plan, route work through controlled workflows, optimize time/resource assignments, react to events, and maintain a complete execution/audit history.

3.2 Primary Function

Plan Work, Automate Processes & Optimize Schedules.

3.3 Source-defined Outputs

- Task plans and executable work items.
- Automated workflows and approval chains.
- Conflict-free schedules and calendars.
- Resource allocation recommendations/assignments.
- Progress, risk, blocker, SLA and scheduling status.

3.4 Business Value

- Reduce manual project and administrative planning effort.
- Standardize institutional processes and improve compliance.
- Shorten approval cycle time and proactively escalate SLA violations.
- Improve resource utilization and reduce timetable/resource conflicts.
- Identify blockers and risks before they materially affect delivery.

4. Design Principles and Boundaries

Principle	E-7 interpretation
Open-source first	Adopt or extend mature open-source intelligence/engine components where they meet most requirements; avoid unnecessary reinvention.
Build the intelligence layer	Expose planning/orchestration/scheduling intelligence through APIs instead of rebuilding a full ERP.
Platform agnostic	Keep domain entities configurable so the same engine can operate in education, HR, healthcare or enterprise settings.
Own the stack gradually	Commercial LLM APIs may be used temporarily, but production should progressively favor low-cost/open-weight models where acceptable.
Multi-tool, lean builds	Use specialized components for planning, workflow and optimization rather than one heavy proprietary platform.
Dual access model	Universal and Embedded modes must use the same core services with different context/data scopes.
Context-aware	Role, active module/entity, tenant context, filters and history should influence plans and actions.
Event-driven	Task, workflow, SLA and schedule events should

	propagate through Kafka/RabbitMQ or equivalent messaging.

Source basis: *IR MAIN DOC pages 1-8 and Engine Architecture pages 1-6 establish these cross-engine principles.*

5. Explicit Goals and Non-Goals

5.1 Goals

- Convert natural-language or structured objectives into validated WBS plans.
- Manage dependencies, priorities, effort estimates, milestones, resource recommendations and execution risks.
- Create and execute configurable approval/business workflows with deterministic state transitions.
- Enforce SLA timers, reminders and escalations with complete audit trails.
- Generate and continuously repair feasible schedules under hard and soft constraints.
- Provide APIs and events for ERP, calendar, communication and analytics integration.
- Maintain tenant isolation, RBAC and organization-scoped data handling in Embedded mode.
- Provide operational dashboards and explainable recommendations without allowing LLMs to directly perform unsafe state changes.

5.2 Non-Goals

Proposed implementation detail: E-7 should not replace the client ERP, HRMS, LMS or calendar product. It should orchestrate and optimize those systems through adapters. It should also not use free-form LLM output as the authoritative state machine, approval decision, SLA clock, or constraint solver.

6. User Roles and Access Model

Role	Typical permissions
Platform Admin	Tenant setup, adapters, model configuration, global policies, audit access.
Organization Admin	Configure workflow templates, resources, calendars, SLA policies and tenant integrations.
Operations/Project Manager	Create goals/projects, approve AI plans, prioritize, assign resources, monitor blockers and reschedule.
Workflow Designer	Build/edit BPMN flows, rules, approval chains, SLA/escalation policies.
Scheduler/Timetable Admin	Maintain constraints, generate schedules, inspect conflicts, lock assignments and publish calendars.
Approver/Manager	Act on approval tasks within authorized workflow states.
Assignee/Staff	View and update assigned tasks, submit evidence, report blockers, acknowledge schedule changes.
Auditor/Compliance	Read-only access to workflow history, rule decisions, SLA events, schedule changes and model recommendations.
API Client/Service Account	Scoped machine-to-machine access for ERP and third-party integrations.

7. Core Operational Modes

7.1 Task Intelligence Mode

Purpose: transform objectives into executable work and maintain project health. Source-defined capabilities include WBS breakdown, priority/dependency detection, effort estimation, resource allocation, deadline/milestone planning, progress tracking, risk/blocker identification, auto task generation and task recommendation.

Recommended intelligence split: LLM for semantic decomposition and explanation; NetworkX/graph algorithms and CPM for dependency validity and critical-path calculations; historical ML (e.g., XGBoost/LightGBM) only after enough quality execution history exists.

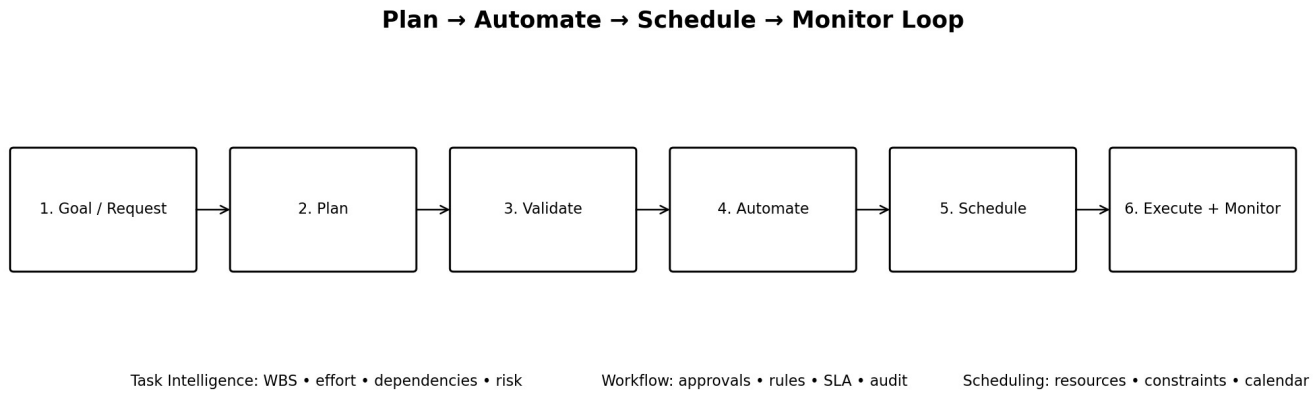
7.2 Workflow Mode

Purpose: execute institutional and administrative processes reliably. Source-defined capabilities include no-code workflow design, approvals, rule engine, SLA management, escalation, automation, analytics, task assignment, notification integration and audit trail. The implementation plan recommends Flowable for BPMN 2.0 visual workflows, Temporal for durable code-first steps, and Drools/Easy Rules for deterministic business rules.

7.3 Scheduling & Resource Optimization Mode

Purpose: compute conflict-free timetables, examination schedules, room/faculty/resource allocation and event calendars. Source documents explicitly state that scheduling is a constraint-optimization problem and should not be LLM-driven. OR-Tools CP-SAT is the primary general-purpose solver; FET is a purpose-built educational timetable option; Timefold is an alternative for advanced rostering.

8. End-to-End Working



1. A user or ERP module submits a goal/request with tenant, role, deadline, constraints and available context.
2. Task Intelligence normalizes the request and proposes a WBS, estimates, milestones, dependencies, risks and resource needs.
3. Deterministic validators check schema validity, dependency cycles, impossible dates, authorization and policy constraints. Human approval is required when configured or when confidence is below threshold.

4. The approved plan is materialized as tasks and linked to a workflow template or generated workflow recommendation.
5. Workflow Engine creates process instances, assigns human/system tasks, applies conditional routing, starts SLA timers and emits events.
6. Scheduling Engine receives tasks/events that need time/resources, builds the constraint model, computes a feasible/optimal assignment, and publishes or proposes the schedule.
7. Execution updates, approvals, task completion, calendar changes and disruptions emit events. Progress/risk and SLA monitors continuously recompute operational state.
8. If a disruption occurs, only the affected schedule neighborhood is re-optimized where possible; workflow state and audit history remain immutable except through defined transitions.
9. Dashboards expose current state; notification responsibilities are routed through the Engagement Engine or configured adapters.

9. Detailed Functional Requirements

ID	Capability	Requirement
FR-TI-01	Goal intake	Accept natural language and structured goals with objective, scope, deadline, constraints, resources and context.
FR-TI-02	WBS generation	Generate hierarchical work packages with IDs, descriptions, owners/roles, estimates, acceptance criteria and parent-child links.
FR-TI-03	Dependency mapping	Propose predecessor/successor relationships and validate acyclic dependency graphs.
FR-TI-04	Effort estimation	Estimate duration/effort with confidence and basis; allow manual override and historical recalibration.
FR-TI-05	Priority	Assign/recommend priority using urgency, impact, dependency criticality and policy rules.
FR-TI-06	Milestones	Create milestone groups and planned dates; track milestone health.
FR-TI-07	Risk/blocker	Detect overdue dependencies, capacity shortages, repeated rework, critical-path slippage and declared blockers.
FR-TI-08	Progress	Track task status, percentage, actual effort, completion evidence and variance to plan.
FR-TI-09	Recommendations	Recommend next best tasks, re-prioritization or mitigation with explanation and confidence.
FR-WF-01	Workflow models	Create/version BPMN-style workflow definitions with human,

		service, timer, gateway and event steps.
FR-WF-02	Approvals	Support single, sequential, parallel, quorum and role-based approvals.
FR-WF-03	Rules	Evaluate deterministic conditions against typed process data; rule outcomes must be auditable.
FR-WF-04	SLA	Track due-at times by state/task, pause/resume rules, business calendars and breach thresholds.
FR-WF-05	Escalation	Create reminder, reassignment, manager escalation or service action when SLA thresholds are crossed.
FR-WF-06	Durable execution	Resume long-running processes after worker restarts without duplicate side effects.
FR-WF-07	Audit	Record every transition, actor, timestamp, input, rule outcome, external call and retry.
FR-SC-01	Constraint model	Represent hard and soft constraints for time, rooms, people, capacity, equipment and policies.
FR-SC-02	Generate schedule	Produce a feasible schedule or return explicit unsatisfied constraints if no solution exists.
FR-SC-03	Optimization	Optimize weighted objectives such as conflict count, preference satisfaction, gaps, fairness and utilization.
FR-SC-04	Incremental rescheduling	Recompute affected assignments after absence/unavailability while minimizing changes to locked/published schedule.
FR-SC-05	Calendar sync	Import/export iCalendar/CalDAV-compatible events and retain mapping to internal schedule entities.
FR-SC-06	Conflict explanation	Explain why an assignment conflicts and which constraint caused infeasibility.
FR-INT-01	Notifications	Emit notification commands/events for E-8 or external messaging adapters.
FR-INT-02	ERP adapters	Support configurable mapping between E-7 canonical entities and external ERP IDs.
FR-INT-03	Events	Publish versioned domain events for tasks, workflows, SLAs, schedules, resources and audits.

10. Non-Functional Requirements

Proposed implementation detail: The source requires secure, scalable and extensible AI capabilities but does not provide numeric SLOs. The following are implementation targets to validate during PoC and adjust after load testing.

Area	Proposed target / requirement
Availability	Core API target 99.9% monthly after production hardening; workflow state persistence must survive process restarts.
Latency	CRUD/API p95 < 500 ms excluding LLM/solver work; asynchronous jobs return job IDs quickly. LLM/solver endpoints expose progress/status.
Scalability	Stateless API workers scale horizontally; solver jobs isolated by queue/worker pool; tenant-aware quotas.
Consistency	PostgreSQL is system of record; state-changing APIs are transactional and idempotent.
Security	OIDC/JWT or API key auth, RBAC/ABAC hooks, TLS, secret manager, encrypted backups, tenant scoping.
Auditability	Every state change and AI recommendation has actor/model/rule/version timestamps and correlation IDs.
Explainability	AI-generated plan items carry rationale/confidence; scheduling conflicts map to explicit constraints.
Portability	Containerized deployment; avoid vendor-specific database extensions unless optional.
Observability	Structured logs, metrics, traces, job/solver duration, workflow failures, SLA breaches and model cost telemetry.
Recoverability	Point-in-time database backups; retry with backoff; dead-letter handling for failed events; workflow replay/recovery strategy.

11. Mandatory Market Research - 5 Comparable Products

The IR Main Document requires market research before technology selection, specifically 4–5 complete open-source or commercial-ready products. The following comparison uses current official product documentation available on 15 September 2026. These products are not exact E-7 equivalents; each covers a major portion of the combined problem, which is itself the key market gap.

Product	Relevant coverage	License / maturity	Strength for E-7	Gap vs E-7
Odoo 19	Projects, tasks, dependencies, milestones, scheduling, activities; broader ERP context	Community edition LGPLv3; mature commercial ecosystem	Strong reference for integrated project/resource UX and dependency-aware planning	Not an AI-first goal-to-WBS engine; deep advanced constraint scheduling and durable BPMN orchestration are not its core role.
ERPNext	Task-driven projects,	GPLv3; mature open-source ERP	Good open-source reference for	No dedicated AI planning layer;

	dependencies, Gantt/Kanban/Calendar, configurable multi-level workflows		project + workflow integration and permissioned ERP data	advanced solver-based timetable optimization is outside its core project/workflow model.
OpenProject	Work packages, hierarchies, dependency relations, Gantt, automatic/manual scheduling, calendars	GPLv3; active open-source project management platform	Excellent reference for work-item semantics, dependency scheduling and progress UX	Limited BPMN-style process orchestration; not an academic constraint-optimization engine; AI planning not core.
Flowable	BPMN/CMMN/DMN engines, REST/Java APIs, human/system process orchestration	Apache 2.0 OSS core; mature BPM platform	Excellent workflow runtime candidate; aligns directly with deterministic workflow requirements	Does not solve WBS intelligence or timetable/resource optimization by itself.
FET	Automatic school/university timetable generation with extensive time/space constraints and exports	AGPLv3; specialized timetabling application, current 7.10.5 (11 Sep 2026)	Strong educational scheduling reference with rich hard/soft constraints	Desktop-oriented specialized scheduler; not a general task/workflow platform and API integration needs adaptation.

Research decision: no single reviewed platform covers the complete E-7 Plan → Automate → Schedule scope while preserving IR Infotech's intelligence-layer/API-first philosophy. The recommended strategy is integration/extension rather than rebuilding every mechanism: reuse a BPMN runtime (Flowable), reuse exact optimization solvers (OR-Tools/FET/Timefold), and build IR-specific task intelligence, canonical APIs, adapters, context handling and cross-mode orchestration.

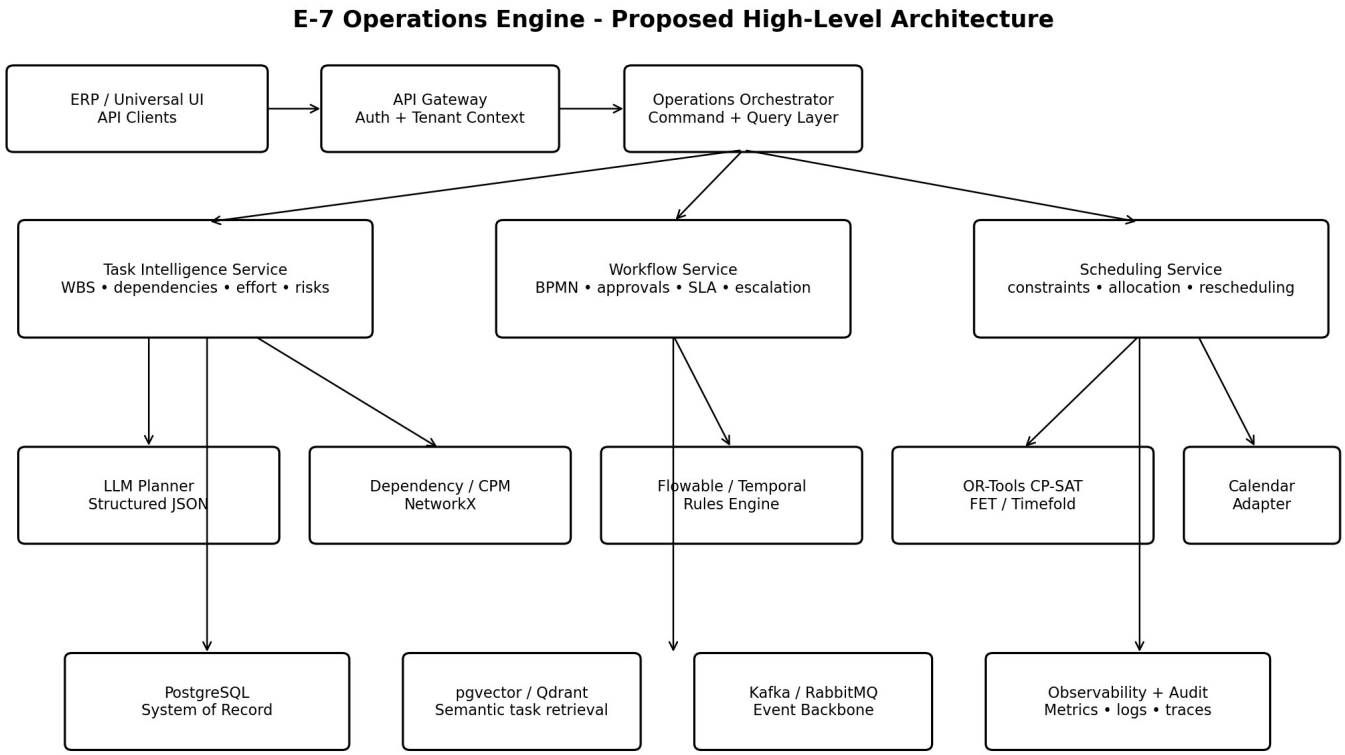
12. Technology Evaluation and Recommended Stack

Source basis: *The E-7 implementation plan names these technologies as recommendations and explicitly allows developer flexibility.*

Layer	Primary	Fallback / alternative	Why
API / service layer	FastAPI (Python)	Spring Boot for Java-heavy services	Fast integration with Python AI/solver stack; Java can host native workflow components.
LLM reasoning	Evaluate Sarvam AI first	Qwen 3, Llama 3.x, Mistral, DeepSeek-R1; commercial APIs only if needed	Low-cost, accuracy-first mandate.
AI orchestration	LangGraph	LangChain or thin custom state graph	Structured multi-step planning with explicit states/tools.

Dependency graph	NetworkX + CPM	Custom DAG algorithms	Deterministic cycle detection, critical path, topological ordering.
Effort ML	XGBoost / LightGBM	Rule/history baselines until enough data	Tabular historical estimates fit well after data maturity.
Workflow	Flowable	Temporal for durable code-first subflows	BPMN visual design + long-running durable execution.
Business rules	Drools / Easy Rules	Custom JSON rule DSL	Keep business transitions deterministic.
Scheduling	OR-Tools CP-SAT	FET / Timefold	Exact/constraint optimization rather than LLM generation.
System DB	PostgreSQL	—	Transactional task/workflow/schedule source of truth.
Semantic retrieval	pgvector or Qdrant	—	Reuse similar tasks/plans/templates and contextual examples.
Events	Kafka or RabbitMQ	—	Event-driven integration and decoupling.
Cache/locks	Redis (proposed)	PostgreSQL advisory locks where enough	Job status, short-lived cache, distributed coordination.

13. High-Level Architecture



13.1 Architectural Layers

- Access layer: Universal UI and embedded ERP clients.
- API gateway: authentication, tenant resolution, rate limiting, request validation and correlation IDs.
- Operations orchestrator: coordinates task intelligence, workflow and scheduling without duplicating their internal logic.
- Domain services: task/plan, workflow/process, schedule/resource, calendar adapter, notification adapter and audit service.
- Intelligence layer: LLM planner, embeddings/semantic retrieval and historical estimation models.
- Deterministic execution layer: dependency graph algorithms, BPMN/rules engines and CP-SAT/constraint solvers.
- Persistence/event layer: PostgreSQL, optional vector store, message broker, object storage if attachments are required.
- Observability/control plane: metrics, logs, tracing, model telemetry, audit and admin configuration.

14. Low-Level Component Specification

Component	Responsibility
Operations API	Canonical REST endpoints, auth/tenant extraction, idempotency keys, versioned DTOs.
Plan Service	Projects/goals/plans/tasks/milestones CRUD, WBS versioning, approval of generated plans.
AI Planner	Prompt/templates, structured schema generation, model routing, confidence, retries, guardrails.

Dependency Engine	DAG validation, cycle detection, topological ordering, CPM/critical path, slack calculations.
Estimation Service	Rule/historical baselines, ML inference, confidence interval, actual-vs-estimate feedback.
Risk Monitor	Scheduled/event-triggered rules for late tasks, blocked critical path, capacity and SLA risk.
Workflow Adapter	Deploy/version BPMN, start/cancel instances, map process tasks to E-7 tasks.
Rule Service	Typed expression/rule evaluation with versioning and deterministic outcomes.
SLA Service	Business-clock calculations, timers, warning/breach events, pause/resume semantics.
Scheduling Service	Constraint model builder, solver workers, feasibility/optimization, lock/freeze and repair.
Resource Service	People/rooms/equipment calendars, capacities, skills, availability and external IDs.
Calendar Adapter	iCal/CalDAV or ERP calendar synchronization, deduplication, time-zone normalization.
Event Gateway	Publish/consume domain events and maintain outbox/inbox reliability.
Audit Service	Immutable append-only logical audit entries with actor, source, previous/new state and correlation.
Analytics API	Operational KPIs, SLA metrics, throughput, utilization, schedule stability and bottlenecks.

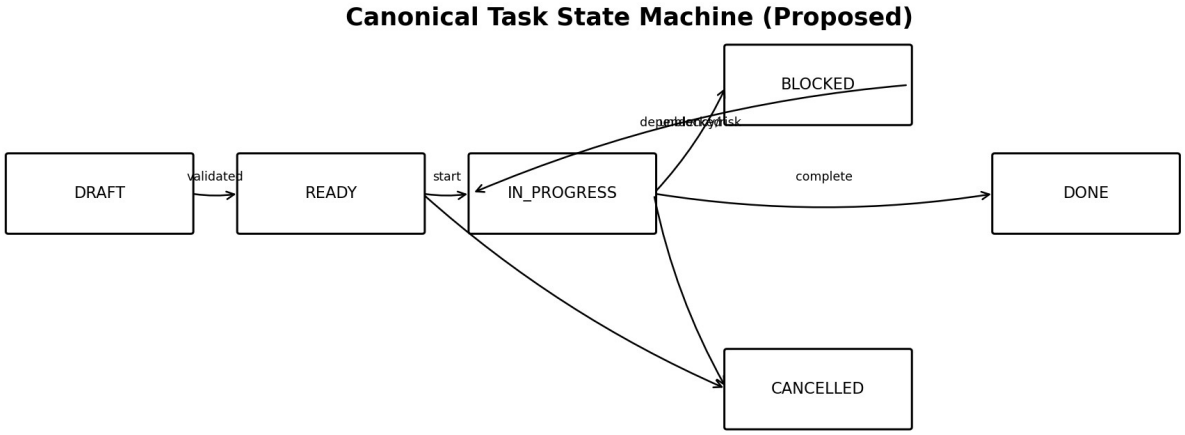
15. Canonical Data Model

Proposed implementation detail: The supplied documents require database design and ERD but do not define entities. The following normalized schema is a proposed canonical model that keeps E-7 platform-agnostic.

Entity	**Key fields / purpose**
tenants	id, name, mode, settings_json, status
users	id, tenant_id, external_id, display_name, role_refs
goals	id, tenant_id, title, description, owner_id, target_date, status, context_json
plans	id, goal_id, version, source, model_id, confidence, approved_by, approved_at
tasks	id, plan_id, parent_id, title, description, priority, status, effort_estimate, start/end constraints, assignee/resource refs
task_dependencies	predecessor_id, successor_id, relation_type, lag, source, confidence
milestones	id, plan_id, title, target_date, status
task_risks	id, task_id, risk_type, severity, probability, signal, mitigation, status
workflow_definitions	id, tenant_id, key, version, engine_ref, model_source, active
workflow_instances	id, definition_id, business_key, state, started_at, ended_at
workflow_tasks	id, instance_id, engine_task_ref, task_id, assignee,

	state, due_at
sla_policies	id, tenant_id, name, duration, warning_offset, calendar_id, escalation_policy
resources	id, tenant_id, type, capacity, skills_json, external_id, active
resource_availability	resource_id, start_at, end_at, availability_type, source
schedule_jobs	id, tenant_id, scope, status, solver, objective_json, started_at, finished_at
schedule_assignments	id, job_id, task/event_id, resource_id, room_id, start_at, end_at, locked, score_components
constraints	id, tenant_id, scope_type, scope_id, type, hard, weight, payload_json, active
calendar_links	id, tenant_id, internal_ref, provider, external_ref, sync_token
domain_events	id, aggregate_type, aggregate_id, event_type, version, payload, created_at, published_at
audit_events	id, tenant_id, actor, action, entity_type, entity_id, before_json, after_json, correlation_id, created_at

16. State Models



16.1 Plan lifecycle

DRAFT → GENERATED → VALIDATED → APPROVAL_PENDING → APPROVED → ACTIVE → COMPLETED / CANCELLED. A regenerated plan creates a new version instead of overwriting an approved plan.

16.2 Workflow lifecycle

Definition: DRAFT → VALIDATED → DEPLOYED → RETIRED. Instance: RUNNING → SUSPENDED / COMPLETED / TERMINATED / FAILED. Engine-native states should be mapped into the canonical E-7 status model without losing detail.

16.3 Schedule lifecycle

REQUESTED → SOLVING → FEASIBLE / INFEASIBLE → REVIEW → PUBLISHED → SUPERSEDED. Published assignments may be locked; incremental repair should minimize movement of locked or unaffected assignments.

17. Task Intelligence Algorithms and Logic

17.1 Goal-to-WBS Generation

10. Normalize objective, domain, deadline, success criteria and constraints.
11. Retrieve similar approved plans/templates if available.
12. Generate a strict structured plan schema (not prose) containing phases, tasks, dependencies, estimates, roles, milestones and assumptions.
13. Run deterministic validation: required fields, parent references, dependency cycles, impossible date ordering, duplicated tasks and policy violations.
14. Ask the model to repair only validation failures using machine-readable error feedback.
15. Score confidence and route to human review based on policy.

Proposed implementation detail: Use JSON Schema/Pydantic validation and require unique stable temporary IDs from the model so dependency references can be validated before database insertion.

17.2 Dependency Management and Critical Path

Represent tasks as a directed acyclic graph. Reject or repair cycles before activation. Use topological ordering for execution readiness. CPM computes earliest start/finish, latest start/finish, float/slack and the critical path. A blocked task on the critical path should have higher operational risk than an equivalent off-path task.

17.3 Effort Estimation

Start with transparent heuristics and historical medians by task type/role. Once enough clean execution history exists, train XGBoost/LightGBM with features such as task category, complexity, assignee skill/experience bucket, historical duration, number of dependencies, rework count, calendar availability and domain context. Store estimate version and confidence so model upgrades remain auditable.

17.4 Risk and Blocker Detection

Proposed implementation detail: Combine hard rules with statistical signals. Suggested signals: overdue predecessor, negative slack, capacity overload, repeated workflow rejection, SLA warning/breach, stale task, estimate variance, resource absence, room/equipment loss, dependency chain growth, and schedule instability. LLMs may summarize risk context, but the triggering signal should remain inspectable.

18. Workflow Execution Specification

18.1 Workflow primitives

- Start/end events, human task, service task, timer, message/event catch, exclusive/parallel/inclusive gateway, subprocess/call activity.
- Typed process variables; no arbitrary code execution from user-authored expressions.
- Versioned definitions; running instances stay on their deployed version unless an explicit migration is performed.

18.2 Approval models

- Single approver, sequential hierarchy, parallel all-of, quorum N-of-M, role-based pool, delegated approval and optional escalation.

18.3 SLA behavior

Proposed implementation detail: Each SLA should define start trigger, duration, business calendar, pause states, warning threshold, breach action and escalation chain. Timer state must persist in the workflow/runtime layer; never rely on an in-memory scheduler only.

18.4 Idempotency and external side effects

Proposed implementation detail: External calls should use idempotency keys and an outbox/inbox pattern. Workflow retries must not send duplicate emails, create duplicate ERP documents or double-book resources.

19. Scheduling and Optimization Model

19.1 Hard constraints

- No person/room/equipment double booking.
- Resource availability and calendar blackouts.
- Room capacity/type/equipment requirements.
- Teacher/faculty qualification or role eligibility.
- Precedence and minimum-gap constraints.
- Exam/class duration and allowed time windows.
- Locked/published assignments that cannot move.

19.2 Soft constraints

- Minimize idle gaps and room changes.
- Prefer requested time slots/resources.
- Balance workload fairly.
- Minimize changes during incremental rescheduling.
- Spread exams/critical activities with preferred gaps.
- Maximize utilization while preserving buffers.

19.3 Objective

A weighted objective can combine penalties for soft-constraint violations while hard constraints remain mandatory. Each schedule job must record objective weights, solver version, seed/settings and final score components for reproducibility.

19.4 Infeasibility handling

Proposed implementation detail: Do not return an empty schedule. Return an INFEASIBLE result containing the smallest practical explanation set: conflicting resources/time windows, unsatisfied hard constraints, and suggested relaxations ranked by impact. A human scheduler chooses whether to relax a constraint.

20. API Design

All endpoints should be versioned under /api/v1 and tenant-scoped by authenticated context rather than trusting tenant IDs supplied by the client.

Method / Endpoint	Purpose
POST /goals	Create operational goal/request.
POST /goals/{id}/generate-plan	Run Task Intelligence and create a draft plan version.
POST /plans/{id}/validate	Run deterministic plan validation.
POST /plans/{id}/approve	Approve and activate plan.
GET /plans/{id}/critical-path	Return CPM and risk summary.
PATCH /tasks/{id}	Update progress/status/actuals/blocker.
POST /workflow-definitions	Create/import workflow definition.
POST /workflow-definitions/{id}/deploy	Deploy a validated workflow version.
POST /workflow-instances	Start a process for a business object.
POST /workflow-tasks/{id}/actions/{action}	Complete/approve/reject a human task.
POST /schedule-jobs	Submit scheduling/optimization job.
GET /schedule-jobs/{id}	Job state, feasibility, score, diagnostics.
POST /schedule-jobs/{id}/publish	Publish assignments after review.
POST /schedules/{id}/repair	Incrementally reschedule affected scope.
GET /resources/availability	Query canonical availability.
GET /analytics/operations	Operational KPIs and health.
GET /audit	Authorized audit search.

20.1 Example - Generate Plan

Request (illustrative):

```
{
 "goal": "Conduct semester examinations for Grade 10",
 "deadline": "2026-12-20",
 "constraints": ["minimum 1 day gap between major exams"],
 "resource_scope": ["faculty", "rooms"],
 "workflow_template": "exam-operations-v1"
}
```

Response should return a job/plan identifier, generated structured tasks, assumptions, confidence, validation warnings and a flag indicating whether human approval is required.

21. Event Model and Integration Contracts

Event	Typical consumers
goal.created	Task Intelligence, analytics
plan.generated	Review UI, audit
plan.approved	Workflow starter, scheduler
task.blocked	Risk monitor, manager dashboard, E-8 notifications
task.completed	Workflow, milestone tracker, scheduler
workflow.task.created	Task inbox, E-8
workflow.sla.warning	E-8, manager dashboard
workflow.sla.breached	Escalation service, audit
resource.unavailable	Scheduler repair queue
schedule.generated	Review UI, calendar adapter
schedule.published	Calendar sync, E-8 notifications
schedule.changed	Affected users, audit, analytics

Proposed implementation detail: Use transactional outbox for state-change + event atomicity. Consumers should deduplicate by event ID and handle schema versions.

22. Integration with the Other Unified Engines

E-7 should remain focused on operations, but it depends on shared services from other engines rather than duplicating them.

Engine	Integration
E-1 Knowledge & Content	Retrieve SOPs/policies/templates that constrain or explain plans and workflows.
E-4 Intelligence & Insights	Consume forecasts/resource demand; provide operational data for analytics.
E-6 Observation	Receive real-time operational/physical events that may trigger workflows or schedule repair.
E-8 Engagement	Send reminders, SLA warnings, approvals and schedule-change notifications over email/SMS/WhatsApp/in-app.
E-9 People Intelligence	Consume staff skills/roles/availability where authorized for resource matching.

23. Security, Authorization and Data Isolation

- Tenant ID is derived from the authenticated principal/API key and applied to every database query.
- Use RBAC for coarse permissions and optional ABAC/policy checks for object/department/context restrictions.
- Workflow actions must verify that the actor is eligible for the exact current task and transition.
- LLM prompts should receive the minimum context required; secrets and unrestricted database data must never be inserted into prompts.
- All model outputs are untrusted input until schema and policy validation completes.
- Audit logs must be append-only at the application level and protected from ordinary user modification.
- External webhooks/API callbacks should be signed and replay-protected.
- Use per-tenant data retention and export/delete policies compatible with customer contracts and applicable privacy law.

24. Observability and Operational Metrics

Metric family	Examples
Planning	plan generation latency, model cost/tokens, validation failures, human correction rate, accepted-vs-edited task ratio
Workflow	instances started/completed/failed, task queue age, SLA warning/breach rate, retries, stuck instances
Scheduling	solve duration, feasibility rate, objective score, conflict count, number of moved assignments in repair
Resources	utilization, over-allocation attempts, availability freshness
Reliability	API p50/p95/p99, error rate, broker lag, DB pool saturation, dead-letter count
Business	approval turnaround, on-time milestone rate, blocker duration, schedule stability, manual planning time

	saved

25. Deployment Architecture

Proposed implementation detail: A practical first deployment is Docker Compose for development and staging, then Kubernetes or equivalent only when scale/operational requirements justify it.

- api-service (FastAPI)
- worker-task-intelligence
- worker-scheduler
- Flowable service / Java runtime
- Temporal server only if durable code-first workflows are used
- PostgreSQL
- Redis (optional)
- RabbitMQ or Kafka
- Qdrant only if pgvector is insufficient
- Prometheus/Grafana + OpenTelemetry/central logs (or organization equivalents)

26. Suggested Repository Structure

```
operations-engine/
  app/
    api/v1/
    core/             # config, auth, tenancy, logging
    domain/           # entities, enums, domain services
    task_intelligence/ # planner, schemas, validators, graph, estimators
    workflow/         # Flowable/Temporal adapters, rules, SLA
    scheduling/       # constraints, solver models, repair
    resources/        # availability/capacity/skills
    integrations/     # ERP, calendar, E-8, webhooks
    events/           # outbox, schemas, consumers
    audit/
    analytics/
    db/               # SQLAlchemy models, repositories, migrations
  tests/
    unit/ integration/ contract/ e2e/ performance/
  workflows/          # BPMN/DMN assets
  solver_profiles/
  scripts/
  docker/
  docs/
```

27. Environment and Configuration

Variable / config	Meaning
DATABASE_URL	PostgreSQL connection.
BROKER_URL	Kafka/RabbitMQ endpoint.
JWT_ISSUER / JWKS_URL	Identity provider configuration.
LLM_PROVIDER / MODEL	Selected task-intelligence model route.
LLM_TIMEOUT / MAX_RETRIES	Inference resilience limits.
FLOWABLE_BASE_URL	Workflow service endpoint.

TEMPORAL_ADDRESS	Optional Temporal endpoint.
VECTOR_BACKEND	pgvector or Qdrant.
CALENDAR_PROVIDER_*	CalDAV/Google/Microsoft/ERP adapter settings as permitted.
SOLVER_MAX_SECONDS	Default optimization time budget.
AUDIT_RETENTION_DAYS	Policy-based audit retention; preferably tenant-configurable.
OTEL_EXPORTER_*	Tracing/telemetry export.

28. Testing Strategy

28.1 Unit tests

- Schema validation, graph cycle detection, CPM, rule evaluation, SLA clock calculations, constraint builder, scoring and state transitions.

28.2 Integration tests

- PostgreSQL transactions, outbox/event publishing, Flowable deployment/execution, scheduler worker, calendar adapter and idempotent external calls.

28.3 Contract tests

- ERP adapters, E-8 notification contract, event schemas and API backward compatibility.

28.4 AI evaluation

Proposed implementation detail: Create a gold set of goals and expected plan properties rather than exact wording. Measure required-task coverage, invalid dependency rate, estimate error, hallucinated entities, correction rate and structured-output validity. Compare Sarvam AI with open-weight fallbacks using the IR 9-dimension evaluation framework.

28.5 Scheduling tests

- Known feasible datasets, deliberately infeasible datasets, regression benchmarks, stress tests and disruption repair scenarios.

28.6 Workflow tests

- Happy path, rejection path, parallel approvals, timeout/SLA, retries, service crash recovery, duplicate callback and permission denial.

28.7 Security tests

- Cross-tenant isolation, broken-object authorization, role escalation, webhook replay, prompt injection through user content, secret leakage and unsafe expression injection.

29. Acceptance Criteria - Consolidated and Measurable

Source basis: *Original acceptance criteria come from the three engine stories. Quantitative thresholds below are proposed testable interpretations.*

Area	Acceptance criterion

Task generation	Valid work items can be generated from a goal and persisted only after schema validation.
Dependencies	No activated plan contains dependency cycles; dependency relationships are returned and visualizable.
Resource recommendation	At least one ranked feasible recommendation is returned when eligible resources exist; reasons/conflicts are shown.
Progress	Task updates reflect in project/milestone health without manual recomputation.
Risks/blockers	Known regression scenarios trigger the configured risk/blocker signal.
Workflow	Configured approval chains execute correctly across sequential and parallel paths.
Rules	Given identical typed inputs and rule version, deterministic rules return identical outcomes.
SLA	Warning and breach events fire at configured business-clock thresholds.
Audit	Every state transition and external side effect has actor/time/correlation metadata.
Scheduling	Feasible benchmark datasets produce no hard-constraint conflicts.
Infeasibility	Infeasible datasets return explicit violated/conflicting constraints rather than a silently partial schedule.
Calendar	Published schedule changes synchronize idempotently and preserve external mapping.
APIs	All documented happy-path and error contract tests pass.
Recovery	Workflow and job state survives service restart without duplicate committed side effects.

30. Phased Implementation Roadmap

This roadmap expands the supplied five phases into implementable milestones. Exact timing depends on team size and integration access.

Phase	Scope	Exit criteria
0 - Research & baseline	Complete 5-product market study, 9-dimension tool/model matrix, sample datasets, domain constraints, API/ERP discovery.	Architecture decision record and PoC test datasets approved.
1 - Foundation	FastAPI, auth/tenancy, PostgreSQL/Alembic, audit, event outbox, CI, OpenAPI. Evaluate Sarvam and fallbacks.	Core API deployed; model benchmark complete.
2 - Task Intelligence MVP	Goal intake, structured WBS generation, validation, DAG/NetworkX, CPM, basic estimates, plan review/approval.	End-to-end goal → approved plan works with regression suite.
3 - Basic Scheduling	OR-Tools model, resources/availability, conflicts,	Benchmark schedules are feasible; explanations available.

	schedule jobs, publish/repair basics.	
4 - Workflow MVP	Flowable integration, BPMN deployment, approvals, rules, SLA, audit and notification events.	Representative approval workflows pass e2e and restart tests.
5 - Education Scheduling Pilot	FET evaluation/adapter or OR-Tools school model, exam/timetable constraints, calendar sync.	Subset of real/synthetic school data scheduled successfully.
6 - Real-time Operations	Event consumers, risk monitor, incremental rescheduling, disruption handling, analytics.	Absence/resource outage triggers controlled repair and notifications.
7 - Hardening	Security, performance, HA/recovery, docs, UAT, deployment automation.	Production readiness checklist and UAT sign-off.

31. Detailed Example - Semester Examination Operations

16. Admin enters: “Plan and conduct Grade 10 semester exams between 1-15 Dec; major subjects need at least one day gap; room capacities and invigilator availability are in ERP.”
17. Task Intelligence creates phases: planning, paper readiness, approvals, room/invigilator preparation, scheduling, exam execution, incident handling and closure. It proposes dependencies and milestones.
18. Dependency engine validates the graph and computes critical path. Human admin approves/edit tasks and assumptions.
19. Workflow starts question-paper approval and readiness flows. Parallel department approvals are allowed; SLA timers escalate overdue approvals.
20. Scheduling service maps subjects, student groups, rooms, invigilators, durations, unavailable periods and gap rules into hard/soft constraints.
21. Solver returns a feasible exam timetable and ranked room/invigilator assignments. Admin publishes it; calendar events are synchronized and E-8 sends notifications.
22. An invigilator becomes unavailable. resource.unavailable event triggers incremental repair for only affected sessions, preserving locked exam times where possible.
23. All changes, approvals, rescheduling reasons and notifications are auditable. Completion updates the plan and milestone status.

32. Failure Modes and Recovery

Failure	**Required behavior**
LLM timeout / invalid JSON	Retry within bounded policy; fall back model if configured; never persist malformed plan; surface error/job status.
LLM hallucinates resource/entity	Reject references not resolvable in canonical resource registry or tenant context.
Dependency cycle	Block activation; return cycle path and repair suggestion.
Workflow worker crash	Durable engine resumes; service actions are idempotent.
External ERP unavailable	Queue/retry with backoff; mark integration state

	degraded; do not fake success.
Solver timeout	Return best feasible solution if policy permits and mark optimality unknown; otherwise keep job unresolved with diagnostics.
No feasible schedule	Return infeasibility report and suggested relaxations; require human decision.
Calendar duplicate callback	Deduplicate using external event ID/version and internal idempotency key.
Message broker outage	Persist outbox; publish when broker recovers.
Cross-tenant reference	Reject before any data access or state change; emit security audit event.

33. Data Migration and Integration Onboarding

24. Inventory external entities: users, roles, projects, tasks, rooms, staff, calendars, workflow states and identifiers.
25. Define canonical mapping and immutable external_id/provider fields.
26. Run read-only import into staging; validate referential integrity and time zones.
27. Backfill availability and active process/schedule state only where source semantics are clear.
28. Reconcile counts and sample records with business owners.
29. Enable incremental sync/webhooks before cutover.
30. Keep rollback path and do not delete source-system data during first production release.

34. CI/CD and Release Controls

- Pull request: lint, type checks, unit tests, schema/API compatibility tests, migration check.
- Integration pipeline: ephemeral PostgreSQL/broker/workflow engine and solver test datasets.
- AI evaluation gate: regression score must not fall below approved threshold on gold set.
- Security: dependency scan, secret scan, container scan and SAST.
- Staging: representative workflows/schedules, migration rehearsal and smoke tests.
- Production: backward-compatible DB migrations, canary or rolling deployment, health checks and rollback plan.

35. Documentation Deliverables

- Architecture Decision Records (ADRs).
- OpenAPI specification and example payloads.
- Canonical data model/ERD and migration guide.
- Workflow authoring guide and BPMN conventions.
- Constraint catalog and scheduling configuration guide.
- Model evaluation report and prompt/schema versioning guide.
- Deployment/operations runbook, backup/restore and incident guide.
- User guide for planner, approver and scheduler roles.
- UAT scripts and acceptance evidence.

36. Product Backlog - Initial User Stories

ID	Story
OPS-001	As an operations manager, I can submit a goal and receive an editable WBS plan.
OPS-002	As a manager, I can approve a plan version and freeze its baseline.
OPS-003	As a project owner, I can see dependencies, critical path and blockers.
OPS-004	As a workflow designer, I can deploy a versioned approval workflow.
OPS-005	As an approver, I can approve/reject only tasks for which I am authorized.
OPS-006	As an admin, I can define SLA warning and escalation policies.
OPS-007	As a scheduler, I can define resources and hard/soft constraints.
OPS-008	As a scheduler, I can generate a conflict-free timetable or receive infeasibility reasons.
OPS-009	As a scheduler, I can lock published assignments and repair only affected items.
OPS-010	As an auditor, I can reconstruct who changed what, when and why.
OPS-011	As an ERP, I can integrate through idempotent APIs and domain events.
OPS-012	As a tenant admin, I can configure engine behavior without exposing another tenant's data.

37. Definition of Done

- Feature meets mapped functional requirement and acceptance test.
- Authorization and tenant isolation are tested.
- API schema documented and versioned.
- State changes and AI recommendations are auditable.
- Failure/retry/idempotency behavior is tested.
- Unit + integration + relevant end-to-end tests pass.
- Observability metrics/logs are present.
- Documentation and migration notes are updated.
- No unresolved critical/high security defects.
- UAT evidence exists for production release scope.

38. Key Risks and Mitigations

Risk	Mitigation
Overusing LLMs for deterministic problems	Strict architectural boundary; typed tools and validation; solvers/rules remain authoritative.
Insufficient historical data for effort ML	Use heuristics/medians first; collect actuals before ML promotion.
Licensing incompatibility	Legal review of AGPL components such as FET if

	integrated/distributed; prefer service boundary or OR-Tools where licensing/product strategy requires.
Complex ERP schemas	Canonical model + adapter layer + contract tests.
Workflow/scheduler divergence	Use shared task/resource IDs and events; one authoritative source for each state domain.
Schedule instability after disruptions	Lock/freeze published assignments and penalize movement in repair objective.
Tenant data leakage to models	Context minimization, provider policy, self-hosted/open-weight path, redaction and tenant-specific config.
Operational complexity from too many tools	Start with minimum viable stack; add Temporal/Qdrant/FET only after demonstrated need.

39. Open Decisions Before Coding

The source documents intentionally leave room for developer choice. The following decisions should be made during Phase 0/1 and documented in ADRs:

- Flowable-only versus Flowable + Temporal.
- RabbitMQ versus Kafka for the initial deployment.
- pgvector versus separate Qdrant.
- Sarvam AI benchmark result and selected fallback model.
- FET integration strategy versus OR-Tools-only school timetable model.
- Canonical rule format: Drools/Easy Rules versus a small internal declarative rule DSL.
- Identity provider and RBAC/ABAC integration contract with the target ERP.
- Calendar providers required for first customer/School ERP.
- Specific production SLOs and data-retention requirements.

40. Final Recommended Implementation Strategy

Build E-7 as a thin orchestration and intelligence layer over proven deterministic engines. Do not build a monolithic project-management application and do not ask an LLM to “run the business process.” The unique IR Infotech value should be the canonical, platform-agnostic intelligence layer that can understand a goal, form an executable plan, select/drive an approved workflow, optimize resource/time assignments, react to real-time events, and expose the result through consistent APIs.

The first production slice should be deliberately narrow: one representative goal-to-plan flow, one approval workflow, one scheduling use case, one ERP/calendar adapter, strong audit/tenancy, and an end-to-end event loop. Once this vertical slice is stable, broaden template coverage, constraints, integrations and AI capabilities.

Appendix A - Traceability to Supplied Acceptance Criteria

Original engine	Source acceptance items	E-7 implementation mapping
Task Intelligence	Auto work items; correct dependencies; resource suggestions; real-time progress; risks/blockers; milestones	FR-TI-01..09, DAG/CPM, resource service, risk monitor, plan/milestone APIs.
Workflow	Successful workflows; correct	FR-WF-01..07, Flowable adapter,

	approvals; expected rules; SLA alerts; audit logs; workflow APIs	SLA service, rules, audit, API contracts.
Scheduling	Timetables; conflict detection; optimized allocation; calendar sync; notifications; APIs	FR-SC-01..06, OR-Tools/FET, calendar adapter, event/notification integration.

Appendix B - Source References

Ref	Reference
S1	IR MAIN DOC(10).pdf - mandatory research process; platform goals; implementation lifecycle; original engine specifications.
S2	Consolidated Engine Directory(5).pdf - E-7 merge, unifying logic, consolidated capabilities and target outputs.
S3	Engine Architecture - IR Infotech AI Ecosystem(4).pdf - core philosophy, dual access model and E-7 detailed modes.
S4	2task_intelligence(1).pdf - Task Intelligence story, business value, deliverables and acceptance criteria.
S5	13workflow(1).pdf - Workflow story, business value, deliverables and acceptance criteria.
S6	19scheduling(1).pdf - Scheduling story, business value, deliverables and acceptance criteria.
S7	42372f20-bcd8-4807-92f4-34c8ad7288c2.pdf - E-7 Comprehensive Implementation Plan.
W1	Odoo 19 official docs: Project, Task Dependencies, Project Templates, Milestones; Odoo Community license documentation.
W2	ERPNext official docs: Project, Tasks, Project Views, Workflows; ERPNext open-source GPLv3 documentation.
W3	OpenProject official docs: Work Packages, scheduling modes, relations, calendar; GPLv3 FAQ.
W4	Flowable official open-source documentation: BPMN/CMMN/DMN engines, REST/Java APIs, Apache 2.0 license.
W5	FET official site: features, AGPLv3 license, current release information and timetable capabilities.

Appendix C - Current Official Web References Used for Market Research

- https://www.odoo.com/documentation/19.0/applications/services/project.html
- https://www.odoo.com/documentation/19.0/applications/services/project/tasks/task_dependencies.html
- https://docs.frappe.io/erpnext/project
- https://docs.frappe.io/erpnext/tasks
- https://docs.frappe.io/erpnext/workflows
- https://www.openproject.org/docs/user-guide/work-packages/

- https://www.openproject.org/docs/user-guide/gantt-chart/scheduling/
- https://www.flowable.com/open-source
- https://www.flowable.com/open-source/docs/oss-introduction
- https://lalescu.ro/liviu/fet/
- https://lalescu.ro/liviu/fet/features.html